cuButterfly: Unleashing Performance of Butterfly Computation with Hierarchical Hybrid Dataflow

ISCA 2026 Internal Working Draft – Anonymous – Do Not Distribute

***Abstract*—Butterfly transforms support scientific computing, machine learning, and cryptography. Efficient execution depends on how their changing pair dependencies are organized under finite storage, bandwidth, and compute resources. Even with a fixed arithmetic core, more local reuse can improve or degrade performance. This recurring trade-off motivates a common method that retains specialized arithmetic while adapting execution organization.**

We present CUBUTTERFLY, which derives local regions, producer–consumer intersections, and readiness from labeled butterfly dependencies. These relations connect spatial expansion and temporal reuse to resource demand. *Hierarchical hybrid dataflow* repeats the construction across storage scopes, composing local residency and boundary exchange under hardware constraints. The construction is shared across arithmetic and workload variants, while their feasible and beneficial organizations can differ.

On A100, fixed-core controls obtain approximately 1.22× organization gains with either of two FFT leaves, with conditional gains also observed for FWHT and NTT. Separate complete-call comparisons obtain geometric means of 1.298×/1.366× over cuFFT for FP32/FP64, including adapted and multidimensional requests, and 1.308× over GPU-NTT across 54 matched requests. Thirty-six short-chain CKKS requests achieve 1.131× over the faster of two native execution choices. FP16 and application losses delimit these benefits. Floating-point errors are reported without a fixed acceptance threshold, and cross-architecture validation remains open.

I. INTRODUCTION

Butterfly computation is a common building block in scientific computing, machine learning, and cryptography. Fourier transforms support fast convolution and long-sequence models [1]–[3]. Number-theoretic transforms provide exact polynomial arithmetic [4], [5], while Hadamard transforms support low-bit model quantization [6]. Learned butterfly factorizations also expose structured linear layers [7]. Their numerical contracts differ, but shared staged pair dependencies motivate a common study of efficient execution.

Mature GPU libraries already optimize both arithmetic and data movement [8]–[11]. Transform generators such as SPIRAL and FFTX also reuse mathematical descriptions to generate implementations [12], [13]. The recurring performance problem is how a compatible core retains values, communicates changing pair groups, and shares finite resources as the workload or target changes. A common method must preserve arithmetic specialization while making these organization choices comparable. Shared dependencies offer that opportunity through a common construction of legal local work and its interfaces.

Arithmetic intensity and Roofline [14] provide a starting point for understanding this variation. Yet matched organizations with the same arithmetic and equal ideal data-reuse intensity attain different pair-update rates on the same device (Figure 1). Organization also changes actual transactions and resource use. We therefore examine **execution organization**: which operations run together, where intermediate values live, and how they reach their consumers. Retention raises ideal reuse but consumes capacity and communication service, and can reduce concurrency. Even abundant parallel work can therefore use hardware inefficiently.

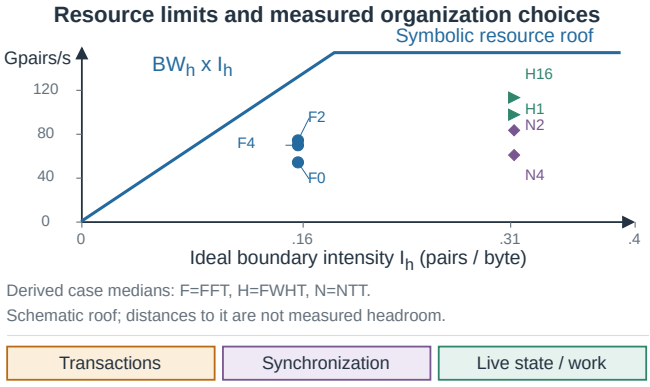


Fig. 1. Measured FFT, FWHT, and NTT organizations attain different rates at equal ideal intensity within each family. The roof is symbolic, not a measured hardware peak.

The effect is measurable independently of core replacement. FFT controls improve complete-call performance by about 1.22× while holding each of two local FFT implementations fixed (Section VI-C). Further FFT, FWHT, and NTT controls show that deeper retention can help or hurt (Section II-B). The question is therefore **whether a common organization method can jointly manage local reuse, communication, and parallel execution across different arithmetic cores, workloads, and hardware.**

The key difficulty is that a local grouping changes both the values that can remain resident and the predecessors needed by the next group. A tile size alone leaves its communication and readiness unresolved. These relationships follow the changing pair structure and can be constructed before choosing a physical layout or core. Their resource demands let different operators compare organizations while retaining specialized arithmetic and workload-dependent choices.

CUBUTTERFLY connects these choices through a **dependency-driven resource characterization**. Stage dependencies determine **dependency-closed regions**; their intersections determine communication and readiness. Spatial

expansion and temporal reuse then characterize live state, transfers, core work, and ready work. Hardware capacities constrain feasibility, while service rates distinguish promising organizations.

We use this characterization to construct **hierarchical hybrid dataflow**. Repeating local retention and boundary exchange across storage scopes handles workloads beyond one local capacity. Tile subgraphs realize these choices by executing dependent operations within a bounded local domain and publishing its boundary values. Boundary layouts account for producer writes and changing successor reads to control contention. Arithmetic, workload, and hardware remain separate inputs whose performance effects are evaluated jointly.

Our two contributions are:

1) A **dependency-driven resource characterization**. Stage-bit closure constructs local regions, and region intersections derive handoff values and predecessor sets. Together with space–time choices, these relations describe storage, communication, core work, and ready work.
2) **Hierarchical hybrid dataflow**. The same construction composes local residency and inter-region handoffs across bounded storage scopes, retaining compatible specialized cores and allowing workload-dependent organizations, including single-band and unit-expansion endpoints.

Subgraph generation realizes these contributions, with search and calibration supporting bounded selection. The A100 evaluation separates fixed-core organization effects from complete-library and application gains. It shows benefits across several arithmetic families, alongside losses and sensitivity to precision and output interfaces. Section VI reports these conditions explicitly; cross-architecture validation remains open.

II. Background and Motivation

The organization question begins with the dependency graph. We first identify its recurring structure and numerical variants, then use measured and analytical examples to derive the requirements for a common method.

A. Butterfly computation and its variants

A butterfly computation combines small groups of values through a sequence of stages. In a radix-two stage, disjoint pairs can execute independently, but the next stage pairs their outputs differently. These changing partners are what make local computation easy to parallelize and the complete computation difficult to organize. For $N = 2^L$, the graph has L stages and $N/2$ pair updates per stage. A stage associated with bit q pairs indices i and $i \oplus 2^q$. The stage sequence defines the dependency envelope, while the pair function supplies arithmetic and coefficients. Batch instances add independent copies of the graph. **Shared structure does not imply interchangeable semantics.**

The family includes complex FFT, finite-field NTT, sum-and-difference FWHT, subset and superset zeta transforms, and products of structured 2×2 stages. One-sided zeta updates

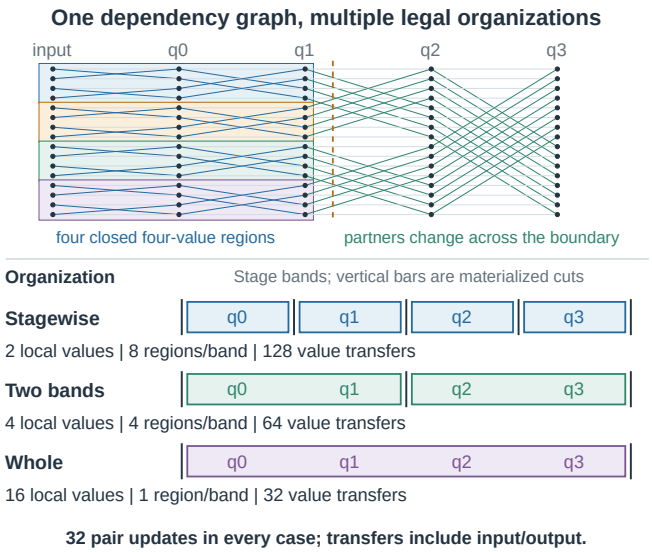


Fig. 2. Grouping stages trades local state and independent regions for fewer transfers in the same Hadamard graph. Counts are analytical.

use a subset of the full pairing dependencies. For example, an FFT pair multiplies by a complex twiddle, an NTT pair performs modular arithmetic, and an FWHT pair uses only sums and differences. A common organization must retain these distinctions rather than replace every pair with one arithmetic implementation. Multidimensional separable transforms compose axis transforms with a shared tensor layout, making inter-axis placement part of the same execution problem.

Cooley–Tukey (CT) and Gentleman–Sande (GS) illustrate another distinction [1], [15]. Their local forms place twiddle multiplication on opposite sides of addition and subtraction. Forward and inverse computation also change coefficients, stage order, permutations, and normalization. Stockham algorithms distribute permutations across passes [16]. Section III-D explains which structural relations survive these changes.

The requested precision, direction, normalization, and input/output layout constrain implementation choices. Internal reordering is legal only when it preserves the request, with adaptation included in execution cost.

The same graph admits different organizations. Figure 2 holds the 16-point Hadamard graph fixed. Stagewise execution exposes eight independent pairs and transfers all values at every stage. Two-stage bands retain four values per region and halve materialized traffic. A whole-transform band minimizes traffic but exposes one independent region per transform. These region counts do not fix worker counts, since workers can cooperate within a region.

Storage scopes give these logical choices physical costs [17]. Worker-local storage, shared storage, and external storage offer different capacities and exchange rates. A generated codelet or imported core such as cuFFTDx [18] must operate within those limits. The preferred grouping therefore depends on both arithmetic service and the surrounding organization.

B. From a resource roof to an execution organization

Changing pair partners makes data retention and parallel execution interdependent. Roofline bounds their performance opportunity but does not map stages, data, and storage together. We examine the same arithmetic under different organizations to derive that mapping.

Reuse raises a resource ceiling. Figure 1 contrasts a symbolic resource roof with measured execution for FFT, FWHT, and NTT. The Roofline opportunity [14] comes from retaining values between dependent stages. For B transforms of length $N = 2^L$, let e be bytes per value and G the number of materialized stage bands. There are $BNL/2$ pair updates. If each band reads and writes the array once, ideal value-only intensity is

$$I_{\text{value}} \simeq \frac{L}{4eG}. \tag{1}$$

At a selected storage boundary, actual intensity is pair work divided by transferred bytes. Its roofline ceiling is the smaller of core throughput and bandwidth times intensity. Coefficients, transactions, cache reuse, and spills change the traffic, while synchronization and execution efficiency determine the distance below this ceiling.

Reducing G raises ideal intensity while changing live state, ownership, communication, and synchronization. The trade-off in Figure 2 therefore affects both the resource ceiling and the execution that approaches it. **The research question is how a parallel organization realizes the available hardware performance.**

Limiting cases expose different organization choices. The H1/H16 points use FWHT FP64 at $N = 2^{20}$ and $B = 1/16$ with two materialized bands. Sixteen times as much independent work raises pair rate from 97.8 to 113.4 Gpairs/s at unchanged ideal intensity. This alone does not distinguish saturation from inefficient organization. It motivates examining how work is served: batch supplies independent work, while the organization determines its storage, communication, and execution costs.

At $N = 2^{20}, B = 16$, F0/F2/F4 use FFT FP64 with local groups of 0/2/4 stages, and N2/N4 use word64 NTT with groups of 2/4. All retain two materialized bands. Pair rate is $BNL/(2T)$, and ideal intensity follows Eq. 1. FFT reaches 54.4, 74.4, and 70.0 Gpairs/s with the same arithmetic and outer boundary. Local reuse helps, but deeper retention does not continue the gain. NTT instead falls from 83.6 to 61.1 Gpairs/s. Thus equal ideal intensity permits different execution rates, and the preferred local organization depends on arithmetic service and resource use.

Two analytical limits extend this reasoning. If a target doubles the applicable resource roof but a serialized exchange keeps throughput unchanged, efficiency relative to that roof halves. Doubling only arithmetic capacity need not raise a bandwidth-limited roof, so the applicable bound matters. Separately, a larger transform can outgrow local storage despite abundant parallel work, requiring smaller resident tile subgraphs and larger-scope exchanges. These are analytical hardware and capacity cases, not measured cross-GPU efficiency results.

The trade-off recurs across butterfly families. The full controls in Figure 11 retain each request's scalar arithmetic primitive and external storage boundaries. Two-stage local grouping improves mean performance for FFT, FWHT, and NTT, while deeper grouping regresses FFT and NTT. Their shared dependencies therefore expose a common trade-off with different preferred choices.

Logical intensity is not measured traffic: grouping also changes exchange, synchronization, and ownership. Matched profiles show increased register use and reduced active warps with deeper grouping. A complementary fixed-core control removes an outer exchange: measured external traffic falls by about 23% despite higher register demand, and complete execution improves (Figure 10). These observations connect the resource trade-off to execution without assigning all gains to one counter. The construction must expose both costs and benefits for each family and workload.

C. Why a hierarchy and a common construction

The observations call for a **joint characterization of resource demand**. Spatial expansion needs concurrent execution, live state, and communication. Temporal reuse retains or moves values for later work at the chosen storage scope; it need not imply repeated external storage accesses.

Stage and data positions provide two allocation axes. Distinct resources can serve successive stages or independent regions; the same resources can instead be reused over either dimension. Their combination forms **hybrid dataflow**. Characterizing the resulting state, traffic, and parallel work makes the trade-off evaluable under a target's constraints. Stagewise execution, a spatial stage pipeline, and a whole-transform local execution are endpoints of this space when their resource demands fit.

Larger transforms can outgrow local storage or bounded compute groups. **Hierarchy repeats the construction under those limits.** Additional bands or recursive refinement preserve locality and handoff rules without fitting the complete intermediate array on chip. Logical depth and materialized groups remain distinct from physical storage levels; refinement adds no hardware capacity.

More batch provides independent work to amortize startup and cover transfers. Realizing that slack still requires lookahead, finite buffers, and available resources. A successor needs its actual predecessor set, not merely a contiguous prefix of producer output. The method must therefore derive both locality and readiness from the graph.

These requirements motivate separate **semantics, workload, and hardware** inputs. Arithmetic supplies functions and labels, length and batch instantiate work, and hardware supplies capacities and service rates. Stage-bit dependencies derive legal local work and handoffs; their resource demands make organizations comparable under the target's constraints.

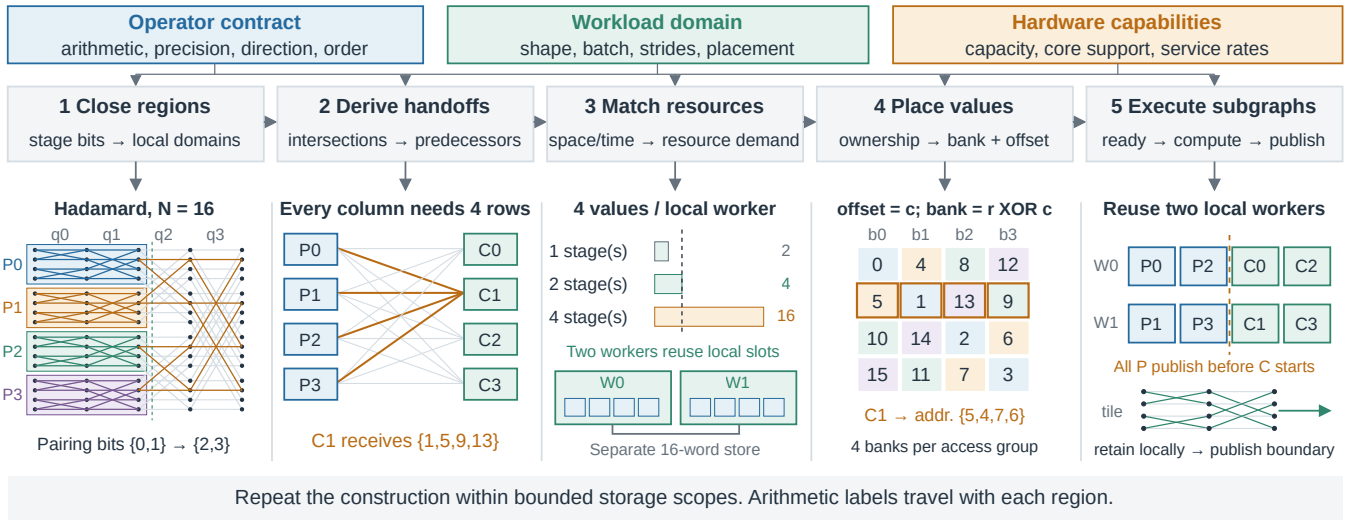


Fig. 3. Five construction steps connect abstract rules to one complete Hadamard organization. The lower row follows the same value indices throughout; capacities and execution order are illustrative, not measured timings.

III. Hierarchical Hybrid Dataflow

We connect **dependency structure, resource demand, and hardware constraints**. The **operator contract** specifies arithmetic, coefficients, direction, normalization, precision, and logical order; the **workload domain** supplies shape, batch, placement, strides, and layout; **hardware capabilities** supply capacities, cores, and service rates. Figure 3 pairs five abstract steps with one 16-point Hadamard example. Closed regions and their intersections determine local work, communication, and readiness. Spatial and temporal choices expose demands; boundary placement and execution realize the organization. Repeating this construction at another storage scope gives the hierarchy.

A. Dependency-closed regions

Local reuse requires a group containing all values needed by its operations. The stage-bit representation constructs such groups directly. **A stage band determines its local data domain.** For a radix-two transform, choose a consecutive stage band and let Q contain its pairing bits. Holding every other index bit fixed gives

$$\mathcal{I}(a, Q) = \{i \mid i \text{ agrees with } a \text{ outside } Q\}. \tag{2}$$

Every pair update in the band changes only a bit in Q and thus stays inside this region. Assignments to the fixed bits partition the array into independent regions of $2^{|Q|}$ values. The band's computation on one region is a **dependency-closed region**. Closure is local to the band, whose inputs and outputs still connect to other bands.

Any ordered stage partition defines regions, including a whole-transform band when capacity and core support permit. Spatial factors may also be one. Band width controls local reuse; regions per worker control worker reuse. Arithmetic labels remain attached. An FFT band may require global-index-dependent twiddles, so a standalone smaller FFT is not automatically a compatible core.

A running example. For a 16-point unnormalized Hadamard transform with stage bits $(0, 1, 2, 3)$, write $i = 4r + c$. Two equal bands give four row subgraphs varying c, then four column subgraphs varying r. Each uses a four-point Hadamard core. Figure 2 places this two-band partition beside stagewise and whole-transform partitions of the full graph. Region membership follows variable bits rather than original contiguity, allowing a representation chosen for physical stores and reads.

B. Communication and readiness

At a band boundary, changing pair partners connect closed regions through their index-set intersections. A producer and consumer with variable-bit sets P and Q share values when they agree on all bits fixed by both. Their shared variable bits $P \cap Q$ determine each nonempty intersection. Producers covering the remaining consumer bits $Q \setminus P$ form its predecessor set. This derives communication without enumerating scalar edges.

Fixing the bits outside $P \cup Q$ groups connected regions into a **boundary packet**. No edge crosses packets at that boundary. Within a packet, every producer and consumer intersect on the full pairing envelope. For complete-region publication, a consumer must therefore await every producer in its predecessor set. One-sided operators or finer publication units can expose finer dependencies.

In Figure 3, each producer P_r contributes one value to each consumer C_c; C_1 needs $\{1, 5, 9, 13\}$ from all four rows. Independent transforms can advance concurrently, and larger transforms may contain several packets. Packet membership is derived at each boundary; it denotes a dependency domain, not a fixed pipeline, buffer, or execution group. Values may

span workers and external buffers. Changing partners can still concentrate accesses on a few banks, so placement must account for both producers and consumers (Section IV-C).

C. Space–time organization and resource demand

Closure defines independent work without fixing an execution group, launch, or pipeline depth. **Hierarchical hybrid dataflow assigns spatial resources and temporal reuse to stages and data.** Successive stages can reuse a worker or occupy different resources; independent regions can execute concurrently or stream through that worker. This organization repeats within regions and across storage scopes.

A worker can retain a small subgraph in local storage, compute groups can use an intermediate shared scope, and larger regions can use external storage handoffs. Refining a parent does not require its entire intermediate array to fit on chip. Hierarchical spatial and temporal mapping has precedents [19]. Here, butterfly closure and intersections determine the legal regions and handoffs.

Characterization separates demand from capability. At each scope, stage-space and data-space factors allocate concurrent stage positions and independent pair work; their temporal counterparts reuse those resources. Each choice has distinct demands:

Stage in space requires concurrent cores, inter-stage transport, coefficients, and live pipeline state.

Stage in time requires retained state or feedback, synchronization, and core reuse.

Data in space requires concurrent workers and state, I/O bandwidth, ports, and coefficient delivery.

Data in time requires sequential tile service, state lifetime, buffering, and fill/drain.

These are organization choices at a selected storage and execution scope. Worker count or compiler unrolling alone does not establish spatial execution.

For example, an ideal fully supplied stage pipeline with U_s stage positions and U_d pair units per position has U_sU_d arithmetic units and consumes $2U_d$ input values per service step. At fixed arithmetic capacity, shifting resources toward stages reduces boundary supply per unit of work while increasing internal transport and pipeline depth. Actual cores, layouts, and resource sharing determine the achievable service rate.

At storage scope h, residency requires

$$\max_t \big(\mathrm{live}_h(t) + \mathrm{aux}_h(t)\big) \le C_h, \tag{3}$$

where live values, coefficients, and buffers are counted in bytes. Compiled register allocation and storage granularity further constrain the mapping. A tile subgraph that does not fit can be refined, rescheduled, or materialized at a larger scope. **Value residency differs from worker persistence.** A finite kernel may keep intermediates local, whereas a persistent worker may exchange them through external storage buffers.

Alongside peak live bytes, the characterization records boundary transfers, core work, and independent ready work. Capacity constrains feasibility; bandwidth, core service, and readiness constrain completion time. Feedback retained at an intermediate scope consumes its service rather than another external pass. Dependency analysis supplies exact logical sizes, intersections, and work; lowering and compilation determine physical ownership, allocation, and instructions; measurements supply service rates. Together they distinguish capacity exclusions from performance choices within the executable space. Logical traffic alone does not predict time.

From dependencies to an organization decision. A band spanning s distinct pairing bits has $BN/2^s$ independent regions. Retaining all values of one region at the chosen scope needs $e2^s$ bytes, before coefficients, buffering, and core scratch space. If G bands each read and write their values once, they transfer $2eBNG$ value bytes at that boundary. These counts connect a wider band to both greater retention and fewer independent regions. Actual transactions and cooperating threads refine these logical demands.

In the 16-point example, a four-value budget after reserving auxiliary storage permits at most two stages per region retained in local storage. Two bands minimize materialized passes under this capacity-only criterion, transferring 64 values instead of 128 for stagewise execution. A sixteen-value budget permits one band and 32 transfers, but exposes only one region per transform; performance still depends on its core mapping. At the four-value budget, a 32-point transform needs at least three such bands. The construction thus extends beyond two bands without increasing local storage. Feasibility, communication, and parallel organization jointly inform the physical choice.

Figure 3 instantiates one feasible choice with two workers, each retaining four values after auxiliary reservations. Each worker reuses its local resources for two pair stages and then for a second region. A separate sixteen-word boundary holds the row outputs before column execution. Stage and tile reuse occur in time; two independent regions use space. The local limit is per worker. This establishes feasibility, while service rates distinguish the feasible organizations.

The construction repeats across bounded storage scopes without requiring every dimension to expand. A larger boundary may use external storage while local regions remain bounded. Section VI-C connects these same counts and constraints to measured FWHT and NTT choices. Before realizing the example, we explain how its structural rules carry to other butterfly variants.

D. Reuse across variants and dimensions

Reuse across stage orders, arithmetic, and tensor interpretations transports the labeled dependency relation, not another transform's numerical output. **Structural correspondence transports organizations.** Bit reversal maps an increasing-stride pairing envelope to a decreasing-stride one, preserving region and boundary relations. Physical layouts must still be evaluated for the transported access pattern.

Figure 4 transports the complete eight-point graph. Reversing the index bits maps pairing bits $(0, 1, 2)$ to $(2, 1, 0)$ while preserving the temporal order of the labeled stages. The highlighted four-value prefix region becomes noncontiguous in index order but remains closed. This correspondence transports

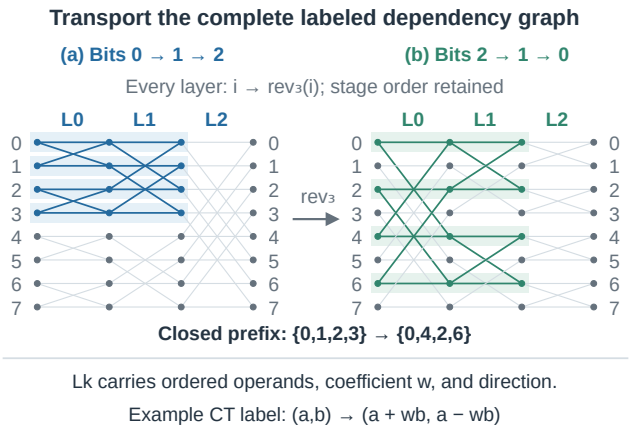


Fig. 4. Bit reversal transports the complete dependency graph and its closed regions while preserving ordered arithmetic labels.

local organizations without arbitrarily exchanging arithmetic stages.

Arithmetic labels move separately. A CT pair computes $(a+wb, a-wb)$ and a GS pair computes $(a+b, w(a-b))$. A GS pair with reciprocal twiddle and one-half scaling inverts the CT pair where those inverses exist. A full inverse also reverses stage order and applies the corresponding permutations. Hadamard changes the local labels to sums and differences, zeta/Möbius exchanges one-sided addition and subtraction, and structured stages require invertible local matrices for inversion. Numerical contracts remain explicit.

For separable power-of-two tensor axes, pairing bits can be labeled by axis and position. Axis-stage composition then uses the same closure and boundary rules on the tensor index domain. Axis order and inter-axis layout become mapping choices subject to the operator's factorization. This describes a shared construction, not a claim that every multidimensional or arithmetic variant already has an efficient lowering.

E. Correct composition

The arithmetic, placement, and scheduling rules must compose into the requested complete operator. A composed execution is correct when it covers each required operation once, each core implements its labels, every handoff preserves value identity, and every read occurs after production and before storage reuse. A topological induction suffices. Correct predecessor values reach the next operation through consistent layouts and valid lifetimes, and its arithmetic produces the required outputs. Thus local arithmetic, boundary layout, and scheduling carry distinct correctness obligations. Tile subgraph execution realizes these rules, controlling residency and the boundary service for producers and consumers to achieve the intended resource trade-offs.

IV. Realizing the Method with Subgraph Execution

A **tile subgraph** realizes dependency-closed regions through a core, value ownership, storage, and handoffs. Common templates supply traversal and exchange for scalar radix-2 FFT, FWHT, and NTT; arithmetic traits supply operations, coefficients, and scaling. Specialized cores may require different boundary realizations.

A. Executing a bounded subgraph

Once inputs are ready, a compute group loads a tile, executes its dependent operations locally, and publishes boundary values. Closure keeps internal producers within the region. In Figure 3, each worker executes two row tiles before the column phase. Each tile retains its two pair stages locally; columns wait for all row outputs. The boundary is reused after the final column read. This schedule needs no cross-band overlap.

The subgraph is the **unit of execution and communication**. Retention can remove outer-storage transfers and submissions, at the cost of local capacity, ports, and synchronization (Section III-C).

A compute group may process several tiles within one kernel, reusing storage after final reads. Only selected tiles and lookahead buffers occupy local storage, not the whole transform. Value residency and worker persistence are independent choices. A submission graph orders kernels; it does not itself retain their intermediates, and subgraph execution does not require one.

B. Successor-aligned writeback

A locally efficient producer may leave outputs inconveniently arranged for its consumer. Rather than add a permutation pass, it can incorporate the next placement into writeback when ownership and stores permit. For each value i, the producer's output and successor's input representations must agree,

$$\lambda_{\text{out}}(i) = \lambda_{\text{in}}(i). \tag{4}$$

Together with compatible ownership and publication, this defines **successor-aligned writeback**.

One construction moves the successor's variable stage bits into low-order address positions, placing its region in a contiguous range. In the Hadamard example, a producer can write logical index $4r + c$ at address $4c + r$, so the next band reads a column contiguously. Identity placement and explicit permutation remain alternatives. Consumer locality alone does not guarantee efficient stores, so the candidate cost includes both transactions and any staging. Figure 5b makes the trade-off explicit. Under an illustrative four-word segment size, the column's addresses $\{1, 5, 9, 13\}$ span four segments; successor-major addresses $\{4, 5, 6, 7\}$ span one. Conversely, one row producer's stores spread from one segment to four. Cooperative staging or different ownership can regroup those stores, but its cost belongs to the boundary. Efficient input supply therefore requires a joint output and input mapping, not a read-only layout choice.

C. Resolving boundary bank conflicts

An aligned layout can still serialize boundary accesses. Successor tiles gather different combinations than producers write. Distinct concurrent addresses can map to the same bank, whose limited ports serialize producer or consumer service

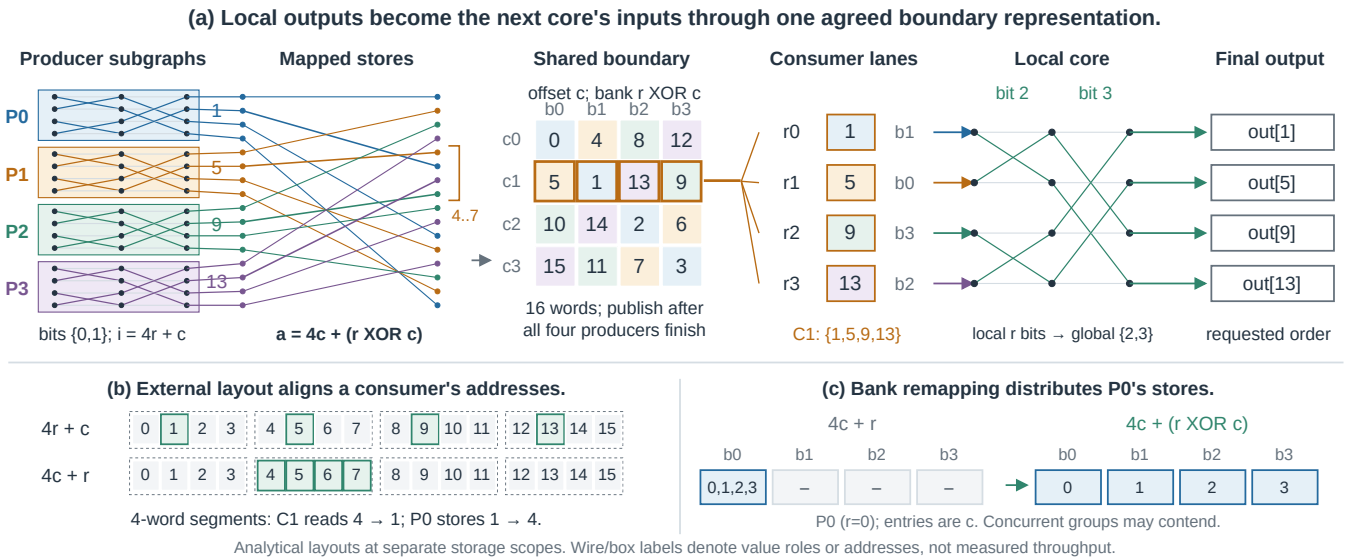


Fig. 5. Boundary placement connects producer outputs to the next core's ordered inputs. (a) Colors track producer value roles through writes and reads by the expanded consumer C_1. (b) External segments and (c) shared banks illustrate distinct access costs; these are analytical layouts, not measured traffic or timing.

even when values fit. Contention between concurrently served groups must also be accounted for.

We address this boundary mismatch with a **successor-aware bank mapping**. The producer and consumer use the same physical placement for each value. Candidate maps combine stage-bit placement with bijective address remapping, including XOR-based *swizzles*. Algorithm 1 gives a bank/offset construction for an aligned contiguous write group and a strided read group. It expresses the mapping in storage-bank coordinates, so the physical address encoding remains specific to the storage scope.

Algorithm 1. Map a producer–consumer boundary.

Given N words, $M = 2^k$ banks, stride $S = 2^s$,
$M \le S \le N$ and power-of-two N.
For each logical word $i \in [0, N)$
$b \leftarrow (i \oplus (i \gg s)) \,\&\, (M-1)$
$o \leftarrow i \gg k$
placement$[i] \leftarrow (b, o)$
Use this placement for producer stores and consumer loads.

For an aligned write group $i = aM + u$, $0 \le u < M$, the high-bit term is constant, so the bank is u XOR a constant. For a read group $i = c + vS$, $0 \le c < S$ and $0 \le v < M$, the bank is $(c \bmod M) \oplus v$. Both are permutations of the M banks for full groups; valid tails remain distinct. Within each offset, the low-bit mapping is also a permutation, giving a bijection over the storage domain. This establishes conflict freedom for these single-word groups on single-ported banks, rather than for arbitrary concurrent accesses.

Figure 5 composes both decisions for the running example. Successor-major placement $4c + r$ gives each column a contiguous range, but a row producer's four stores reach the same bank. For a square tile of power-of-two side P with P single-ported banks and one word per value, retain the successor offset c and permute its banks:

$$\lambda(r, c) = P\,c + (r \oplus c). \tag{5}$$

This is the preceding bank construction with the row and column roles exchanged. XOR by either fixed coordinate is a permutation. Each row store group and each column load group therefore reaches every bank once, and the map is bijective. This guarantee concerns each access group; concurrent groups can still compete for bank service. In Figure 5a, all four producer outputs reach this boundary directly through their mapped stores. P_0 uses banks $(0, 1, 2, 3)$, while C_1 reads values $\{1, 5, 9, 13\}$ at addresses $\{5, 4, 7, 6\}$ and banks $(1, 0, 3, 2)$. Consumer lane r selects bank $r \oplus 1$, restoring the local core's ordered inputs; final results follow the requested output order. The write and read maps agree on each value, avoiding an intervening reorder. Figure 5c contrasts the producer bank accesses before and after this remapping. On GPUs this is a shared-memory bank swizzle [20].

Conflict analysis uses actual transactions, value width, and bank capacity; vectors and broadcasts require separate checks. Outer-storage layouts require alignment, transaction-count, and coalescing checks. Thus one XOR formula need not suit every core or scope. Section VI-D separately measures external layout trade-offs and local bank mapping with fixed arithmetic cores.

D. Residency, publication, and finite buffers

Correct placement still needs readiness and safe reuse. A buffer slot is acquired before writing, published after its values become visible, and released after its final required read. Tail tiles preserve this lifecycle. More slots trade lookahead against live state and synchronization.

Within-band pipelining and cross-band overlap are separate choices. Figure 6 contrasts cross-band readiness with

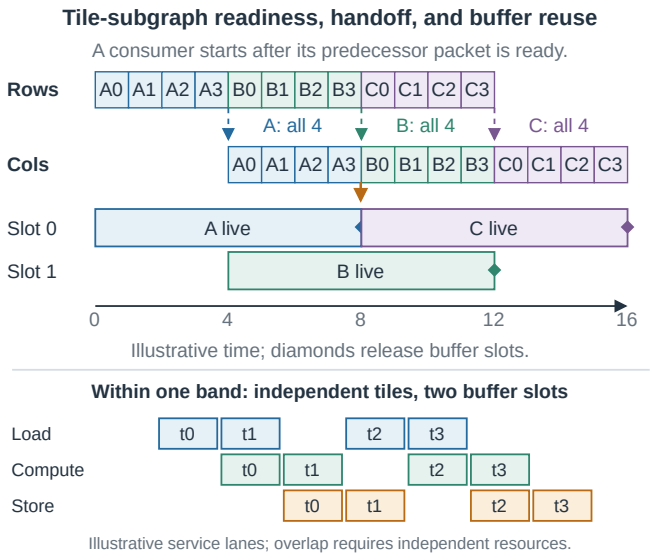


Fig. 6. Top: cross-band readiness for independent Hadamard transforms; C reuses slot 0 after A's final read. Bottom: within-band load/compute/store pipelining with two slots. Durations and lanes are illustrative.

two-slot load/compute/store overlap within a band. The latter can operate despite a barrier before the next band. Cross-band overlap additionally requires complete predecessor sets, free buffers, and execution resources. Once all Hadamard rows publish, their columns may overlap another transform's rows. Batch supplies independent work, but neither guarantees nor requires cross-band overlap.

E. Composing physical services

The characterization specifies resource demands. A **service model** combines them with measured services to estimate startup, repeated work, and tail costs for one compute group within a stage band,

$$T_{\text{group}} = T_{\text{start}} + R\,T_{\text{step}} + T_{\text{tail}}. \tag{6}$$

Here R counts successive tile-service steps in the modeled group. A step may serve several tiles under the chosen mapping; its service contract specifies that grouping. Terms depend on core, tile, layout, and storage scope. Band completion composes its groups under the available concurrency. Bulk execution sums these band times and boundary costs without recounting launches or tails already included in measured services.

For a total of K tiles in a repeatable pipeline, the corresponding view is

$$T_{\text{pipe}} \approx T_{\text{fill}} + (K-1)\,\text{interval} + T_{\text{drain}}. \tag{7}$$

Dependencies, buffer reuse, and shared resources limit the interval; overlapped groups may lose their isolated service rates. Independent work amortizes fill/drain and can cover latency, but cannot exceed resource throughput. This model ranks realizations, without proving optimality.

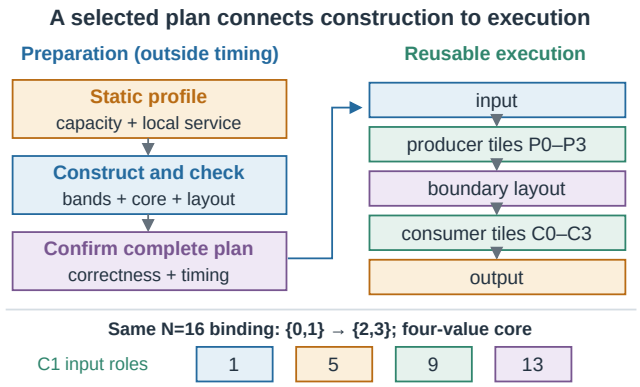


Fig. 7. Preparation constructs and checks subgraph plans, then the selected plan controls local execution and boundary exchange.

V. Kernel Generation and Selection

Generation and selection turn the legal organizations into reusable plans. Figure 7 follows Figure 3's inputs to residency, layouts, concurrency, and handoffs. The complete device path defines warm execution; preparation, including compilation, is excluded.

Compatible realizations. Generation combines bands, regions, cores, storage, layouts, and bulk/pipelined schedules while preserving arithmetic, precision, direction, normalization, order, shape, and placement. Adaptation costs remain included; missing compatible lowering remains a gap. Static checks enforce dependencies and capacity; compilation exposes local allocation, intermediate and temporary storage, and launch requirements. Numerical checks precede timing. Failed checks remain recorded but untimed. Cache reuse requires matching semantic, workload, hardware, and compilation assumptions.

Service estimates. Services are keyed by arithmetic, core, tile shape, layout, and resource regime, separating full and remainder tiles when necessary. Piecewise linear interpolation stays within compatible measured regimes; missing coverage requires measurement or an explicit gap. Composition respects predecessors, buffers, and shared-resource contention: shared execution and memory capacity can lengthen concurrent service intervals. Submission costs are added only when absent from measured services. Estimates guide preparation; search instantiates the resource characterization rather than constituting a contribution.

Bounded selection and confirmation. Finite-budget selection uses bounded enumeration or model-guided exploration; explicit evolutionary tuning adds multistart neighborhood mutation. Multidimensional selection measures whole candidates before refining axes. Compatible caches and calibration avoid repeated preparation. All modes share semantic/resource constraints and retain complete organization costs. Complete-plan timing confirms selection, separately from static ranking. Organization comparisons distinguish explicit interventions from searched plans, retaining actual mappings, tuning conditions, and confirmation protocols. Finite search does not establish global optimality.

VI. Evaluation

We compare complete operators and applications with external libraries, isolate fixed-core organization effects, and explain their resource costs. Comparisons retain their populations and timing boundaries.

A. Experimental setup

Platforms and workloads. The main comparison uses A100 80GB PCIe (SM80), CUDA 12.4, and PyTorch 2.5.1. Its 66 requests cover FFT, NTT, FWHT, subset/superset zeta, and structured butterflies at ranks 1, 2, 3, 4, and 8. Precision, direction, normalization, order, shape, batch, placement, and strides define a request. An independent cyclic NTT study covers 90 requests with 32/64-bit words, multiple moduli, both directions, $N \in \{256, 65536, 1048576\}$ and $B \in \{1, 7, 16\}$. Nine public FP16 selections span $N = 2^{12}$–2^{18} and $B = 4$–2048. Further BF16 comparisons are deferred; cuFFT Xt's BF16 storage and mixed internal arithmetic differ from our native-BF16 core.

Historical local-grouping, cross-core, and low-precision boundary controls use A100 40GB; the independent service and tuned-endpoint studies use 80GB. Fixed-core FFT controls use 80GB, with an independent 40GB FP32 replication. The independent external-layout and local-bank controls also use 80GB. Cohorts retain distinct device/build/protocol identities. All platforms are A100; PRO 6000 remains unmeasured.

Numerical and timing contracts. All 29 main-study integer and 90 cyclic NTT requests agree exactly with their references. The 37 floating requests have finite outputs and retained error distributions. Their CPU references use FP64, including for FP32 outputs; these higher-precision FP32 diagnostics remain distinct from the same-precision FWHT and explicitly rounded FP16 controls below. We report floating errors without a fixed acceptance threshold, so their performance results remain exploratory. Relative L_2 error divides the difference norm by the reference norm. Main-study FP64 FFT errors span 2.45×10^{-16}–4.96×10^{-16}; structured butterflies span 6.82×10^{-8}–9.15×10^{-8} in FP32 and 0–2.97×10^{-16} in FP64 against those references. FP16 scale controls repeat one seeded transform scaled by 2^{-6} across batches and backends. Random weights and short-chain cryptography establish neither task quality nor production security.

Operator timing includes required conversions and output materialization, using ten warmups and the median of seven CUDA Graph replays, each containing twenty calls, normalized per call. Search, compilation, allocation, capture, and checking are outside this warm execution boundary. Application ratios match boundaries within each cohort: FNO and QuaRot use host-wall medians, filtering and Poisson use device-event medians, and CKKS/bootstrap use device-event sample means. We do not pool these application protocols. Profiler timings do not replace ordinary timings; Graph replays are not independent process trials, and reported ranges are not confidence intervals. Near-parity ratios do not establish significance.

TABLE I
Operator ratios retain their matched populations. N/A/X counts native, adapted, and axis-composed baseline calls.

Operator	Baseline	Pairs	GM	Faster	N/A/X
Main 66-request study					
FFT FP32	cuFFT	11	1.298×	9/11	5/4/2
FFT FP32	VkFFT	11	1.190×	5/11	9/0/2
FFT FP64	cuFFT	6	1.366×	5/6	3/3/0
FFT FP64	VkFFT	6	1.354×	6/6	6/0/0
FWHT FP32	DaoFHT	10	1.474×	9/10	0/10/0
NTT w64	GPU-NTT	5	1.188×	4/5	0/5/0
Independent FP16 public DEFAULT study					
FFT FP16	cuFFT	9	0.854×	1/9	–
FFT BF16	cuFFT	0	–	–	–
Independent cyclic NTT study (90 requests)					
NTT w32	GPU-NTT	18	1.400×	18/18	–
NTT w64	GPU-NTT	36	1.265×	33/36	–

TABLE II
FWHT gains depend on the matched output interface. Ranges span three process GMs; DEFAULT is a separate single-process check.

Output interface	Fixed-mapping replays	Public DEFAULT
Native return	1.013–1.031×	1.009×
Supplied buffer	1.320–1.330×	1.324×

B. Performance comparisons

Question 1: How competitive are the generated operators? Figure 8 and Table I report matched baseline/cuButterfly latency ratios. The 66 requests yield 49 external pairs; some have multiple competitors, others no matched baseline. Boxes span workloads, not uncertainty. Missing pairs are not counted as wins.

FFT FP32 and FP64 achieve geometric means (GM) of 1.298× and 1.366× over cuFFT. Complete calls include requested layout and normalization adaptation. Native cuFFT subsets give 1.034× over five FP32 pairs and 1.225× over three FP64 pairs; adapted subsets give 1.290× over four FP32 pairs and 1.524× over three FP64 pairs. Independent rank-4/8 axis-composition replays retain 1.925/2.748×. The gains therefore combine local execution and organization around the cores. For the eleven FP32 requests, relative L_2 error against the common CPU FP64 reference ranges from 7.90×10^{-8} to 2.98×10^{-7} for cuButterfly and to 2.94×10^{-7} for cuFFT, with the same minimum. These per-request ranges are neither acceptance bounds nor confidence intervals.

Output contracts affect the FWHT comparison. The main-study 1.474× against DaoFHT includes copying its returned tensor to a supplied buffer. Table II independently compares both interfaces after contiguous-access specialization, with the same ten requests and mappings. Both sides retain twenty independent outputs in the native-return Graph; both reuse one caller buffer in the supplied-output Graph. Each comparison matches output lifetime, but subtracting the two interfaces would not isolate copy cost.

Public DEFAULT selects the same ten mappings. All sixty paired pre/post-specialization outputs are byte-identical. A

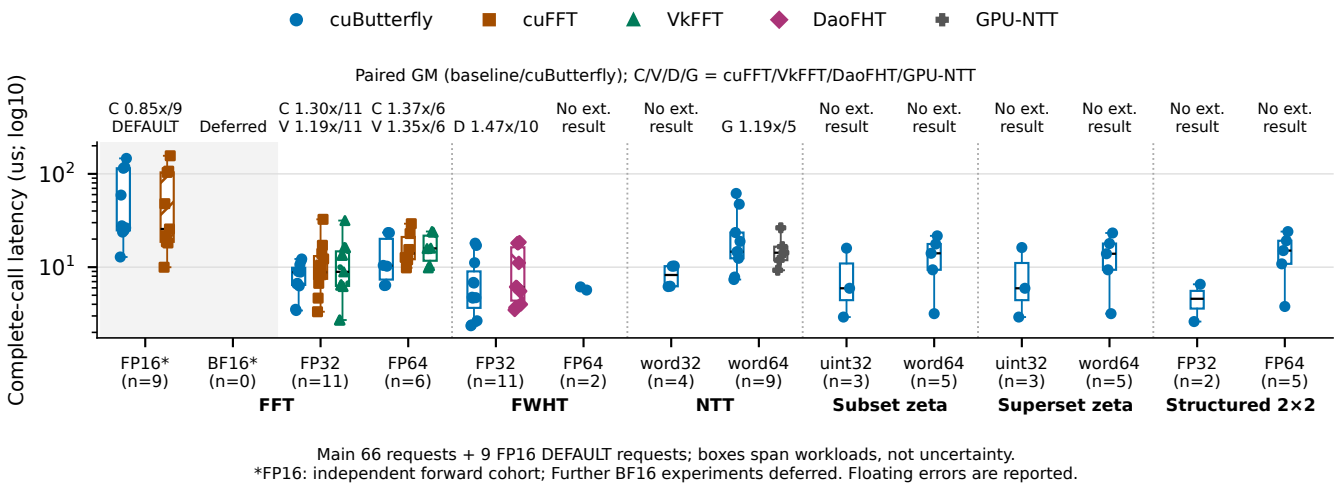


Fig. 8. Frozen complete-call comparisons span operator families and precisions. The main-study DaoFHT calls include supplied-output copying; the independent FWHT interface study is reported separately in Table II. FP16 uses nine public selections, while further BF16 comparisons are deferred.

separate CPU FP32 audit gives maximum absolute error 4.88×10^{-4} and relative L_2 error 1.50×10^{-7} for both backends on these inputs; historical FP64-reference diagnostics remain separate. Native-return results are near parity, with individual losses, while the supplied-output interface benefits more. FP64 FWHT, zeta, and structured butterflies lack matched GPU baselines. Their separate 34-request CPU study uses two processes on a shared host and a different timing boundary.

Finite-field range and batch. The five main-study word64 NTT pairs give 1.188×; three-trial replay gives 1.175×. The independent cyclic study obtains 54 exact GPU-NTT pairs with GM 1.308×, including 51 ratios above one. Its 18 word32 pairs give 1.400× and 36 word64 pairs give 1.265×. Comparator modulus limits (2^{30} for word32 and 2^{61} for word64) exclude 36 requests, which retain exact cuButterfly results without external ratios. Three word64 $B = 1$ pairs yield 0.972–0.999×. Eighteen pairs were measured together; 36 use unchanged cuButterfly timings and separately collected baselines on the same GPU/protocol.

Limits of the aggregate gains. Independent three-process confirmations yield 0.838× for FP64 FFT at $N = 2^{20}, B = 16$ and 0.939× for FP32 FWHT at $N = 2^{15}, B = 32$ on their recorded builds. The nine-request FP16 cohort yields 0.854× against cuFFT, ranging from 0.733 to 1.067×. Only $N = 2^{18}, B = 32$ wins, at 146.53 versus 156.31 μs. DEFAULT and bounded measured selection choose the same mappings in all nine requests. Against the same explicitly rounded CPU FP16 reference, relative L_2 errors span 0.00193–0.00260 for cuButterfly and 0.00173–0.00198 for cuFFT. Maximum pointwise relative errors on nonzero references reach 16.54/7.43, respectively; small normwise errors do not bound these tails. Both produce finite outputs; matching storage does not equate internal arithmetic. An independent eleven-request 40GB cohort gives GM 0.858× (0.739–1.088×), outside the 80GB aggregate, with unseen-request ranking unvalidated because independent stage calibration is missing.

Question 2: Do operator gains reach complete applications? Matched applications share inputs, precision, keys, and weights. Timings include surrounding work and required copies, excluding reusable setup [21]. Figure 9 retains losses and separate cohorts, including their Graph/eager protocols.

FP32 periodic Poisson at $(N, B) = (1024, 4), (2048, 4), (2048, 16)$ gives 1.082–1.107× over Torch/cuFFT and 1.097–1.125× over CuPy. Twenty event samples include input refresh, both transforms, frequency multiplication, and output copies. Changed/restored-input and output-lifetime checks pass; CPU-reference relative L_2 error is below 5.7×10^{-7}. Fixed 31^2 filtering on $1024^2/2048^2$ images, $B = 16$, caches the spectrum and matches $1080^2/2100^2$ padding. Two independent repeat processes give 1.068–1.069/1.048× over CuPy. Recomputing the spectrum is a different contract and wins only three of nine earlier requests.

CKKS crosses key switching, square/relinearize/rescale, square/rescale/key switching, and rotation with $B = 1, 8, 32$: methods I/II at $N = 16384$ and method I at $N = 65536$. Holdout Graph timing follows loop/stage-batch selection. All 36 paths produce identical ciphertexts, with GM 1.428× over the native loop. The control below separates integration gains. Small-parameter regular bootstrap at $N = 4096$, $B = 1/8$ gives approximately 1.03× over HEonGPU from five samples per backend. Both cryptographic cohorts use `sec_level=none`; bootstrap source/binary correspondence is unverified, limiting reproducibility of that comparison.

FNO at $64^2/128^2$, $B = 4$, gives 1.030/1.056× in Graph mode and 0.955/0.957× in eager mode, including copies and synchronization [22]. QuaRot's composed conversion/FWHT/scaling path gives 0.957–0.968× at prefill/decode 128/8, $B = 1/4$. An existing core reading FP16, computing/scaling in FP32, and writing FP16 gives 1.004–1.005×/1.015–1.017× for the complete 7B model in both process orders; all nine output steps match exactly [23].

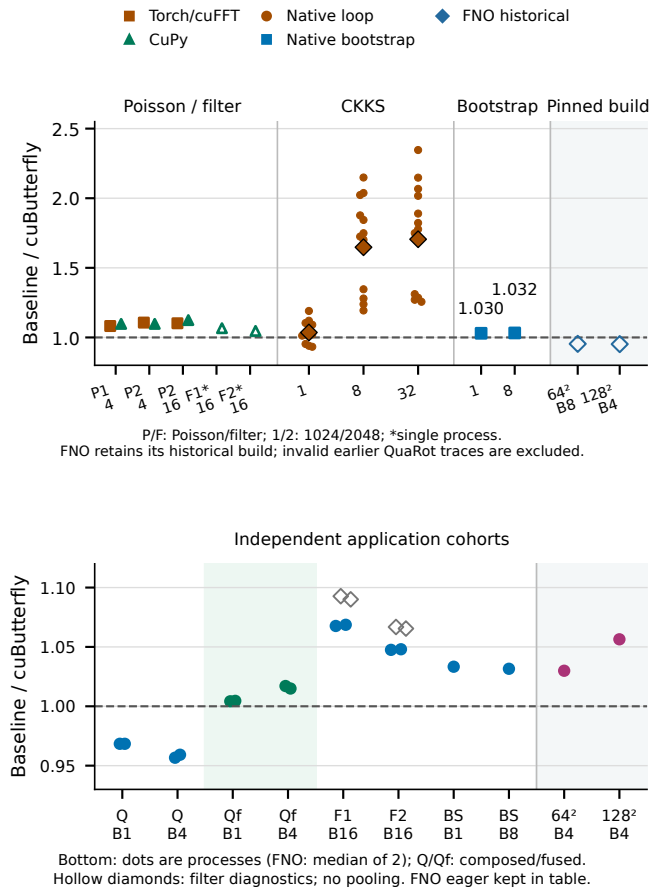


Fig. 9. Application gains vary with workload and integration. The upper panel retains the main and historical cohorts; the lower panel shows independent repeats and composed/fused QuaRot paths. Cohorts are not pooled.

TABLE III
CKKS GAINS PERSIST AFTER MATCHING BATCH INTEGRATION.

B	Requests	External Loop	Integration controls Batch	Faster native
1	12	1.036	1.290	1.036
8	12	1.648	1.183	1.182
32	12	1.705	1.182	1.182

Core selection also changes, so these small gains cannot be attributed to fusion alone. Earlier QuaRot traces without valid random-state preparation and output checks are excluded. Application benefits depend on these execution contracts.

Matched application integration. The CKKS control gives native NTT cores the same gather/scatter and Graph organization as our stage-batch path, without assuming an upstream batch API. Table III compares the native loop, matched batch, and the faster native alternative per request.

Across the same 36 requests, GM falls to 1.218× over matched native batch and 1.131× over the faster native alternative. Thirty-three ratios exceed one; the three losses occur at $B = 1$, while both larger batch groups retain about 1.182×. This is an integration control, not fixed-core attribution.

C. Ablations

Question 3: What does organization contribute with a fixed core? Library and application comparisons combine arithmetic and organization choices. To isolate subgraph execution (Section IV-A), we cross register and cuFFTDx Thread 16-point FFT leaves with external exchange between radix-16 steps and a resident 256-value prefix spanning two such steps (eight radix-2 stages). The resident prefix uses shared storage; later steps stay unchanged. The four cells separate organization from core substitution.

Six FP32 workloads, four cells, and three trials produce 72 checked rows. The resident organization improves all six workloads for both leaves, giving GM 1.217/1.221×, versus 1.042/1.046× from core substitution at fixed organization. Independent replication gives organization gains of 1.226/1.231×. Ownership, storage, launches, and wrappers change while the leaf and requested transform stay fixed. At $N = 256, B = 1$, the organizations launch 16 and 256 threads. This tests Section IV-A's complete organization, without isolating residency or internal overlap.

A stronger endpoint bounds the organization gain. The FP64 control at $N = 65536, B = 16$ fixes two 256-point factors, four local transforms per tile, and 16 elements per thread. Both cores share the factor template and writer-aligned handoff. Three fresh processes on a second A100 80GB device accompany the matched profiles in Figure 10. At four consecutive tiles per compute group, one-step prefetch reduces register/cuFFTDx Block latency from 125.901/134.554 to 65.403/71.987 μs, or 1.925/1.869×. Against the stronger one-tile synchronous endpoints (75.325/80.404 μs), the gains are 1.152/1.117×. Depth two regresses to 80.118/93.143 μs. Relative L_2 error against CPU FP64 is below 7.3×10^{-16}. These are same-device internal controls, kept separate from the earlier trials and external-library comparisons.

Reuse across arithmetic families. Figure 11 fixes scalar radix-2 FFT/FWHT FP64 and word64 NTT while varying local stage grouping. It covers 18 workloads at $N = 2^{16}, 2^{20}$ and $B = 1, 4, 16$, three organizations, and three process trials, for 162 observations. Two-stage groups give GM 1.196/1.106/1.199× for FFT/FWHT/NTT; groups of up to four stages give 0.932/1.103/0.816×. Here the two outer bands remain fixed; local grouping changes exchanges within them. Section III-C's retention/service trade-off recurs within a storage scope, with family-dependent benefits.

One construction, different organization choices. The same dependency rule yields 4096 regions per eight-stage band for FWHT FP32 at $N = 2^{16}, B = 16$, and 16,384 per ten-stage band for word64 NTT at $N = 2^{20}, B = 16$. FWHT regions contain 256 values (1 KiB), and NTT regions contain 1024 (8 KiB). Both use two disjoint-bit bands: within each transform, producer–consumer pairs share one value, requiring every producer before any consumer.

FWHT groups one/two/four suffix regions per compute group. Suffix group counts fall from 4096 to 2048/1024, while source-derived shared-storage lower bounds rise from

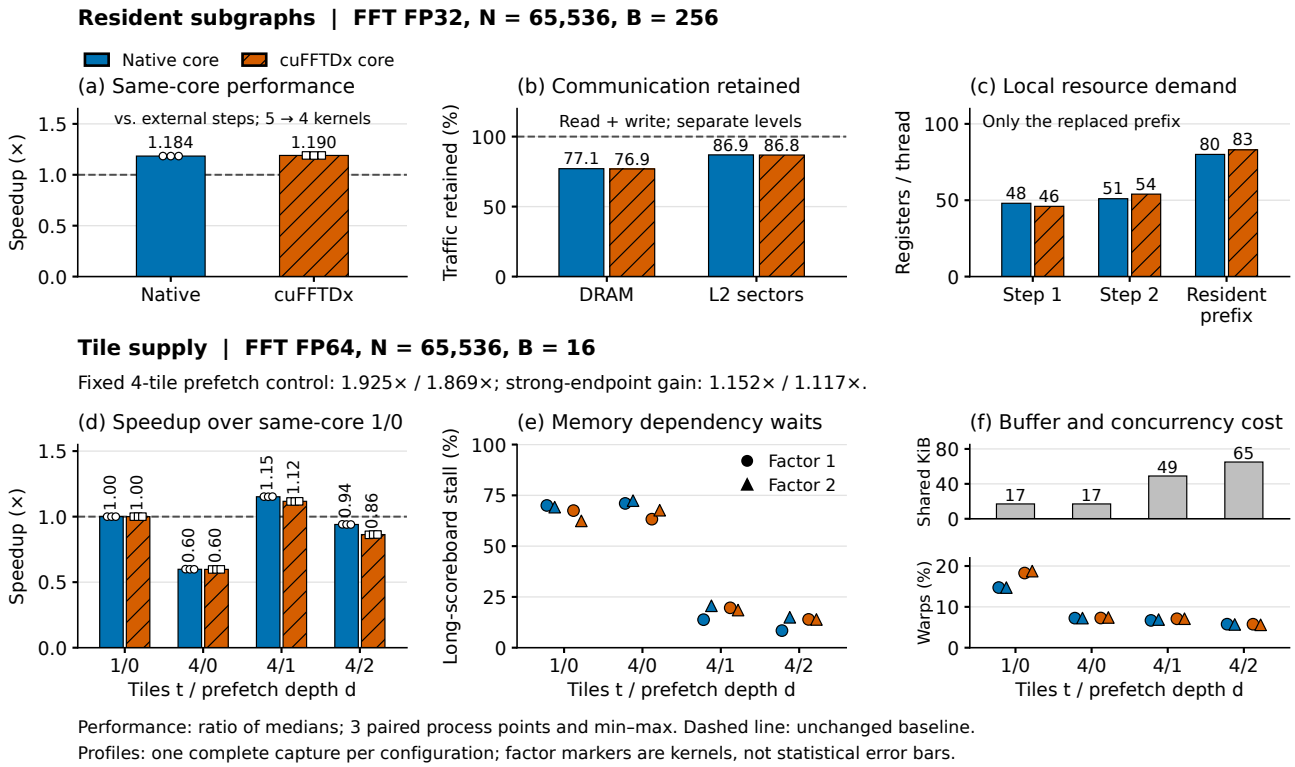


Fig. 10. Organization gains coexist with local resource costs. (a–c) Resident prefixes versus external steps; (d–f) tile supply with the stronger 1/0 reference. Speedup bars are ratios of three-process medians; points and ranges retain paired process ratios. Traffic uses the same-core external-step reference; factor points represent separate kernels. t/d denotes consecutive tiles per compute group/prefetch depth; (f) shows allocated shared storage per compute group and active-warp fractions.

TABLE IV
THE COMMON CONSTRUCTION ADMITS DIFFERENT MEASURED CHOICES. TIMES AND PAIRED RATIOS ARE THREE-PROCESS MEDIANS; "SAME" REUSES ONE MAPPING.

Leaf	Conv. μs	Ext. μs	Ratio
FWHT r2	60.447	45.875	1.318
FWHT r4	59.894	43.878	1.365
NTT shared-r4	1585.398	same	1.000
NTT warp-r2	3706.778	same	1.000

1 to 2/4 KiB. NTT instead traverses one/two/four independent transforms per group, using one cooperative launch and external handoffs. Traversal grouping does not merge dependency packets. Logical value traffic stays at 16/512 MiB for FWHT/NTT across these choices; neither quantity is measured traffic or simultaneous on-chip storage.

Each fixed leaf retains its conventional endpoint under matched tuning: FWHT tunes 128/256 threads in both domains; NTT fixes 256. Conventional/extended pools contain 2/6 and 1/3 candidates, respectively, under a six-candidate ceiling. All execute successfully, so capacity alone does not choose among them. Candidates and resource expectations are frozen before screening, followed by three-process confirmation with order reversal.

FWHT selects 256 threads/four columns over 128 threads/one column, giving 1.315–1.318× and 1.365–1.367× across processes. Both NTT domains retain the mapping, timed once per process; unity denotes identity (Table IV). NTT checks are full-batch exact; all twelve FWHT candidates match CPU FP32 over 2^{20} outputs. Fixing an internal leaf does not fix execution geometry: FWHT changes threads and columns, and NTT's shared-radix4 traversals use different wrappers. These are complete-organization outcomes, not isolated grouping effects. NTT retains its endpoint without a measured benefit from changing it.

Optional mechanisms have costs. In a factor-streamed FFT control at $N = 256, B = 16384$, slice overlap changes native/imported-core latency from 127.560/126.331 to 132.346/130.957 μs. A separate pipeline-batching control loses to bulk in all 27 comparisons; at $N = 4096, B = 64$, times are 45.636 versus 29.266 μs. Writer-aligned layout also loses in three tested pairs, whereas greater resident depth wins in three. These test conditional applicability. Eight-candidate searches disabling overlap, resident fusion, or successor layout show small FFT/FWHT differences and NTT differences of at most 2.9%. Two rejected unrestricted mappings leave unequal accepted pools, limiting attribution. Finite comparisons establish no global optima.

Composition reuse. Across 138 coset, negacyclic, and RNS requests, replaying existing cyclic mappings per modulus gives GM 1.691× over internal public defaults (range 1.051–2.996×). Each request first geometrically averages its two

input-pattern ratios. All 552 measurements across two input patterns and two implementations pass exact checks. Both core and organization may change, so this supports composition reuse rather than fixed-core attribution or joint RNS search. Default runs first, and Graph replays are not independent process trials.

D. Analysis

Question 4: Which resource costs explain organization choices? The ablations establish which organization changes help. NCU counters and NSYS timelines examine why, relating traffic, storage, and concurrency to the resource characterization through matched internal controls.

Subgraph execution removes a measured exchange. Figure 10(a–c) profiles the complete FP32 call at $N = 65536, B = 256$ with both fixed leaves. The resident prefix removes one external exchange and reduces kernel count from five to four, including the common input permutation. Ordinary latency falls from 1.414/1.431 to 1.194/1.203 ms, or 1.184/1.190×. Measured DRAM read-plus-write volume falls by 22.9/23.1%, while L2 sector volume falls by 13.1/13.2%. Logical boundaries, cache traffic, and external-memory traffic remain distinct. The resident kernel uses 80/83 registers per thread versus 46–54 for the separate steps. Reduced communication wins despite higher local demand; residency remains confounded with ownership, wrappers, and launches. CPU FP32 relative L_2 error is below 4.0×10^{-7} for all four implementations.

Tile prefetch changes supply rather than the stage boundary. Figure 10(d–f) aligns complete-call performance with supply and resource costs. All eight FP64 variants retain two factor kernels. At fixed four-tile traversal, emitted asynchronous copies accompany a reduction in per-kernel long-scoreboard stalls from 63.3–72.4% to 13.8–20.6%, and increased issue activity, while active-warp fractions remain only 6.7–7.1%. Depth two lowers these stalls further but increases allocated shared storage from 49 to 65 KiB per compute group and reduces active-warp fractions to 5.6–5.8%; the complete call becomes slower. Neither waiting nor reuse alone determines performance. Single-call NCU profiles use uncontrolled caches and clocks; counters are diagnostic, while unprofiled timings supply speedups. NSYS confirms launch scope, not overlap inside a kernel.

From resource demand to complete-plan service. Table V combines dependency-derived regions/traffic, compiled storage/registers, and independent service curves for three FFT/FWHT requests, using 34 service keys and 117 clean probes. Median relative errors are 0.565% over 68 group holdouts and 0.429% over 15 complete-plan confirmations. Removing complete-plan-fitted fallback parameters leaves predictions unchanged. However, the final composition artifact postdates confirmation, so this is retrospective validation, not certified prospective prediction. It does not cover NTT.

Unlike the prospective measured choices in Table IV, these estimates test composition from independent local services. Neither study establishes unseen-hardware prediction.

TABLE V
INDEPENDENT SERVICES REPRODUCE COMPLETE-PLAN TIMES RETROSPECTIVELY. G COUNTS ORDERED STAGE BANDS; SHARED STORAGE IS IN KIB AND TIMES ARE IN MS.

Request (N/B)	G	$\text{SMEM}^{\max}$	Retro.	Confirm
FFT64 $2^{20}/16$	2	64	1.0651	1.0727
FWHT32 $2^{16}/16$	7	4	0.0450	0.0449
FWHT32 $2^{15}/32$	1	32	0.0186	0.0184

More local reuse need not improve execution. Figure 11 retains two outer bands (8+8 stages at $N = 2^{16}$ and 10+10 at $N = 2^{20}$), changing local shared exchange under fixed arithmetic. At $N = 2^{20}, B = 16$, each of the two kernels is configured with 16,384 compute groups of 256 threads. Two-stage leaves change measured DRAM traffic little. Four-stage leaves increase register demand and reduce active-warp fractions from roughly 49% to 24% for FFT, 97% to 48% for FWHT, and 98% to 12–25% for NTT; barrier-stall percentages also rise. Fewer exchanges can accompany greater pressure and waiting. Shared-access counts and instruction-level attribution are not available for these controls.

Overlap must offset fragmentation. The factor-streamed FFT variants both launch three kernels, with measured L2 traffic differing by less than 0.2%. The imported core uses fewer registers, yet improves the complete call by only about 1%. NSYS observes overlap in the bulk/pipeline control, but pipeline launches eight kernels instead of two. Concurrency alone does not imply faster completion. Similarly, six stage-

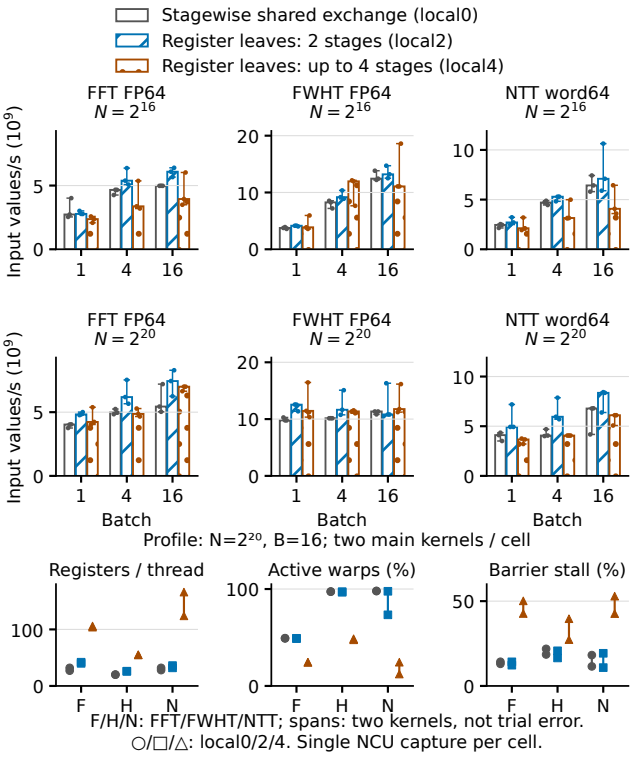


Fig. 11. Local grouping changes throughput and resource pressure. Top: medians, process points, and ranges. Bottom: two kernel records per organization at $N = 2^{20}, B = 16$; their spans are not trial uncertainty.

grouping controls reduce launches from 12 to 2 or 18 to 3; capture traffic includes setup and replay and must not be read as ordinary complete-call traffic.

Boundary placement has scope-dependent effects. External-boundary controls (Section IV-C) fix scalar FFT/FWHT FP32 cores, $(N, B) = (2^{16}, 16), (2^{20}, 4)$, equal two-band partitions and 256-thread groups. Median paired speedups from three processes are 0.973–1.020× for direct consumer-oriented stores over strided boundaries. In the smaller FWHT case, suffix load sectors per request fall from 31.9 to 4, while prefix stores rise from 4 to 32: better reads can cost more writes.

A cuFFTDx control fixes partition 8+12 at $N = 2^{20}$, changing only prefix shared staging from linear to XOR placement. At $B = 4/16$, median paired process speedups are 1.247/1.248×. Prefix load/store bank conflicts fall by about 94/83%, with 80 registers per thread in both variants. Complete timings include both computation kernels and the final transpose. Outputs are byte-identical, with CPU FP32 relative L_2 error below 3.3×10^{-7}. The two controls isolate different storage scopes.

Boundary improvements coexist with local-core limits. At FP16 $N = 4096, B = 256$, matched two-band implementations retain packed arithmetic instruction counts. Joint ownership/writeback changes test boundary service (Section IV-C), without isolating a swizzle, and reduce store sectors per request from 32 to 4 in both bands and total warp instructions from 34.32 to 11.48 million. Barrier-stall fractions fall from 75.2/55.8% to 10.5/14.0%. The complete call improves from 126.26 to 43.16 μs, although the 34.20 μs single-band alternative and 11.11 μs cuFFT call remain faster. Eightfold fewer store sectors do not imply eightfold fewer external-memory bytes; some shared-memory bank conflicts increase.

Load also changes the preferred organization. FP16 radix-4 single-/two-band times are 13.11/9.27 μs at $N = 4096, B = 4$ but 33.59/39.83 μs at $B = 256$; at $N = 8192, B = 32$ they are 32.36/19.35 μs. Single-band execution and unit expansion must remain available. In the independent public FP16 cohort, paired imported local transforms use one pass at $N = 4096/8192$ and two factors at larger lengths, with writer-aligned handoffs and grouped, prefetched tiles at larger batches. Factor kernels use 96–128 registers/thread and about 15–17 active warps, versus cuFFT's 32–48 registers and 35–59 warps. Resource pressure accompanies a local-core efficiency gap, although two factor passes beat cuFFT's three passes at $N = 2^{18}, B = 32$.

Preparation remains a separate cost. A ten-request public DEFAULT FWHT check takes 43.2 minutes to prepare plans on the shared host. Reuse amortizes preparation outside warm execution. This is not an installation benchmark; the evidence notes retain measurements and protocols.

VII. Related Work

Table VI contrasts transform generation, specialized execution, and resource mapping. cuButterfly connects these concerns through dependency-derived regions, handoffs, and resource demands.

TABLE VI
Representative systems expose different organization choices.

System	Ops./ object	Decomposition basis	Organization / resources
FFTW [2]	FFT, DCT/DST	Recursive plans	Codelet composition; empirical planning
VkFFT [9], [24]	FFT, DCT	Grouped strided transforms	Layout choices
GPU-NTT [4], [5]	NTT	Merge / Four-Step	Locality; kernel count; block shape
TensorIR [25]	Tensor ops.	Block signatures	Outer transformations; tensorization
Welder [26]	DNN graphs	Tile connections; shape propagation	Hierarchical reuse; traffic / footprint
Timeloop [19]	DNN layers	Workload / mapping separation	Spatial / temporal tiling; resource costs
cuButterfly	FFT, NTT, FWHT	Stage bits; closed regions; intersections	Stage / data reuse; live state, traffic, core / ready work

Transform generation and specialized execution. FFTW combines codelets with empirical planning; SPIRAL/FFTX generates transforms and surrounding computation from algebraic descriptions [2], [12], [13]. VkFFT generates GPU implementations with grouped strided transforms and layout choices. GPU-NTT optimizes locality, kernel count, block size, and shape [4], [5], [9], [24]. FlashFFTConv combines tensor-core FFT decomposition, batch/hidden tiling, and memory-aware fusion [3]. HadaCore maps FWHT chunks to warps and vectors to threadblocks with shared-memory transposes [27]. These systems optimize both arithmetic and organization.

Tensor compilation and tile scheduling. TVM separates computation from scheduling [28]; Ansor searches hierarchical schedule sketches [29]; ROLLER constructs hardware-aligned tiles [30]. TensorIR provides block signatures, outer transformations, and tensorization [25]. Welder connects producer/consumer tiles and schedules reuse through traffic and footprint estimates [26]. These precedents establish core/organization separation and dependency-aware tile connections. Our specific addition is a reusable butterfly construction: pairing bits derive closed regions for each stage band, and region intersections derive exchanged values and predecessor sets. Repartitioning recomputes these relations; compatible cores and layouts reuse them while arithmetic labels preserve the numerical contract.

Hierarchical dataflow and positioning. Eyeriss analyzes data movement under controlled resources; Timeloop separates workload, architecture, and hierarchical spatial/temporal mappings [19], [31]. Our construction supplies these mapping principles with butterfly regions and handoffs, from which live state, traffic, core work, and ready work are characterized at each scope. GPU blocks share resources, so logical stage expansion does not assume dedicated pipeline hardware.

Fixed-core controls and the FWHT/NTT walkthrough test reuse and conditional performance effects (Section VI-C). The

evaluated claim concerns the implemented butterfly space, without asserting greater expressiveness or better optimization than general-purpose compilers.

VIII. Conclusion

cuButterfly derives local regions, handoffs, and resource demands from butterfly dependencies, composing them through hierarchical hybrid dataflow under finite hardware resources. A100 controls show fixed-core organization gains and different preferred choices across families. Library and application comparisons establish practical benefits and limits under their stated numerical and timing contracts. Cross-architecture performance remains unvalidated.

References

[1] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex Fourier series," *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965.

[2] M. Frigo and S. G. Johnson, "The design and implementation of FFTW3," *Proceedings of the IEEE*, vol. 93, no. 2, pp. 216–231, 2005. [Online]. Available: https://www.fftw.org/fftw-paper-ieee.pdf

[3] D. Y. Fu, H. Kumbong, E. Nguyen, and C. Ré, "FlashFFTConv: Efficient convolutions for long sequences with tensor cores," in *International Conference on Learning Representations*, 2024. [Online]. Available: https://arxiv.org/abs/2311.05908

[4] A. Ş. Özcan and E. Savaş, "Two algorithms for fast GPU implementation of NTT," Cryptology ePrint Archive, Paper 2023/1410, 2023. [Online]. Available: https://eprint.iacr.org/2023/1410

[5] A. Ozcan, A. Javeed, and E. Savas, "High-performance number theoretic transform on GPU through radix2-CT and 4-step algorithms," *IEEE Access*, vol. 13, pp. 87 862–87 883, 2025. [Online]. Available: https://doi.org/10.1109/ACCESS.2025.3570024

[6] A. Tseng, J. Chee, Q. Sun, V. Kuleshov, and C. De Sa, "QuIP#: Even better LLM quantization with Hadamard incoherence and lattice codebooks," in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235, 2024, pp. 48 630–48 656. [Online]. Available: https://proceedings.mlr.press/v235/tseng24a.html

[7] T. Dao, A. Gu, M. Eichhorn, A. Rudra, and C. Ré, "Learning fast algorithms for linear transforms using butterfly factorizations," in *Proceedings of the 36th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 97, 2019, pp. 1517–1527. [Online]. Available: https://proceedings.mlr.press/v97/dao19a.html

[8] NVIDIA, "cuFFT library documentation," https://docs.nvidia.com/cuda/cufft/index.html, accessed September 16, 2026; experimental binary versions are recorded separately.

[9] VkFFT Project, "VkFFT: GPU fast Fourier transform library," https://github.com/DTolm/VkFFT, official implementation and documentation; accessed September 16, 2026.

[10] GPU-NTT Project, "GPU-NTT: Merge and 4-step number theoretic transforms on GPUs," https://github.com/Alisah-Ozcan/GPU-NTT, official implementation; accessed September 16, 2026. Measured adapter versions and semantic scope are recorded separately.

[11] Dao AI Lab, "Fast Hadamard transform in CUDA, with a PyTorch interface," https://github.com/Dao-AILab/fast-hadamard-transform, official implementation; accessed September 16, 2026.

[12] M. Püschel, J. M. F. Moura, J. R. Johnson, D. Padua, M. M. Veloso, B. W. Singer, J. Xiong, F. Franchetti, A. Gačić, Y. Voronenko, K. Chen, R. W. Johnson, and N. Rizzolo, "SPIRAL: Code generation for DSP transforms," *Proceedings of the IEEE*, vol. 93, no. 2, pp. 232–275, 2005.

[13] F. Franchetti, D. G. Spampinato, A. Kulkarni, D. T. Popovici, T. M. Low, M. Franusich, A. Canning, P. McCorquodale, B. V. Straalen, and P. Colella, "FFTX and SpectralPack: A first look," in *IEEE 25th International Conference on High Performance Computing Workshops (HiPCW)*, 2018, pp. 18–27. [Online]. Available: https://www.spiral.net/doc/papers/hipc_2018.pdf

[14] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.

[15] W. M. Gentleman and G. Sande, "Fast Fourier transforms," in *AFIPS Fall Joint Computer Conference*, 1966.

[16] T. G. Stockham, Jr., "High-speed convolution and correlation," in *AFIPS Spring Joint Computer Conference*, 1966.

[17] NVIDIA, "NVIDIA Ampere GPU Architecture Tuning Guide," https://docs.nvidia.com/cuda/ampere-tuning-guide/index.html, accessed September 16, 2026.

[18] ——, "cuFFTDx: Execution methods," https://docs.nvidia.com/cuda/cufftdx/api/methods.html, accessed September 16, 2026.

[19] A. Parashar, P. Raina, Y. S. Shao, Y.-H. Chen, V. A. Ying, A. Mukkara, R. Venkatesan, B. Khailany, S. W. Keckler, and J. Emer, "Timeloop: A systematic approach to DNN accelerator evaluation," in *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2019, pp. 304–315.

[20] NVIDIA, "CUDA Programming Guide: Shared-memory bank swizzling," https://docs.nvidia.com/cuda/cuda-programming-guide/04-special-topics/async-copies.html\#shared-memory-bank-swizzling, accessed October 29, 2026.

[21] HEonGPU Contributors, "HEonGPU: Gpu-accelerated fully homomorphic encryption library," https://github.com/Alisah-Ozcan/HEonGPU, open-source implementation; accessed September 21, 2026. Application source revision is pinned separately in the experiment manifest.

[22] NeuralOperator Contributors, "NeuralOperator: Learning in infinite dimensions," https://github.com/neuraloperator/neuraloperator, open-source implementation; accessed September 21, 2026. Application source revision is pinned separately in the experiment manifest.

[23] QuaRot Contributors, "QuaRot: Outlier-free 4-bit inference in rotated LLMs," https://github.com/spcl/QuaRot, open-source implementation; accessed September 21, 2026. Application source revision is pinned separately in the experiment manifest.

[24] D. Tolmachev, "VkFFT—a performant, cross-platform and open-source GPU FFT library," *IEEE Access*, vol. 11, pp. 12 039–12 058, 2023. [Online]. Available: https://doi.org/10.1109/ACCESS.2023.3242240

[25] S. Feng, B. Hou, H. Jin, W. Lin, J. Shao, R. Lai, Z. Ye, L. Zheng, C. H. Yu, Y. Yu, and T. Chen, "TensorIR: An abstraction for automatic tensorized program optimization," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. ACM, 2023, pp. 804–817. [Online]. Available: https://arxiv.org/abs/2207.04296

[26] Y. Shi, Z. Yang, J. Xue, L. Ma, Y. Xia, Z. Miao, Y. Guo, F. Yang, and L. Zhou, "Welder: Scheduling deep learning memory access via Tile-graph," in *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, 2023, pp. 701–718. [Online]. Available: https://www.usenix.org/conference/osdi23/presentation/shi

[27] K. Agarwal, R. Astra, A. Hoque, M. Srivatsa, R. Ganti, L. Wright, and S. Chen, "HadaCore: Tensor core accelerated Hadamard transform kernel," arXiv:2412.08832, 2024. [Online]. Available: https://arxiv.org/abs/2412.08832

[28] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, H. Shen, M. Cowan, L. Wang, Y. Hu, L. Ceze, C. Guestrin, and A. Krishnamurthy, "TVM: An automated end-to-end optimizing compiler for deep learning," in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, 2018, pp. 578–594. [Online]. Available: https://www.usenix.org/conference/osdi18/presentation/chen

[29] L. Zheng, C. Jia, M. Sun, Z. Wu, C. H. Yu, A. Haj-Ali, Y. Wang, J. Yang, D. Zhuo, K. Sen, J. E. Gonzalez, and I. Stoica, "Ansor: Generating high-performance tensor programs for deep learning," in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, 2020, pp. 863–879. [Online]. Available: https://www.usenix.org/conference/osdi20/presentation/zheng

[30] H. Zhu, R. Wu, Y. Diao, S. Ke, H. Li, C. Zhang, J. Xue, L. Ma, Y. Xia, W. Cui, F. Yang, M. Yang, L. Zhou, A. Cidon, and G. Pekhimenko, "ROLLER: Fast and efficient tensor compilation for deep learning," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022, pp. 233–248. [Online]. Available: https://www.usenix.org/conference/osdi22/presentation/zhu

[31] Y.-H. Chen, J. Emer, and V. Sze, "Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks," in *Proceedings of the 43rd Annual International Symposium on Computer Architecture*, 2016, pp. 367–379. [Online]. Available: https://eems.mit.edu/wp-content/uploads/2016/04/eyeriss_isca_2016.pdf